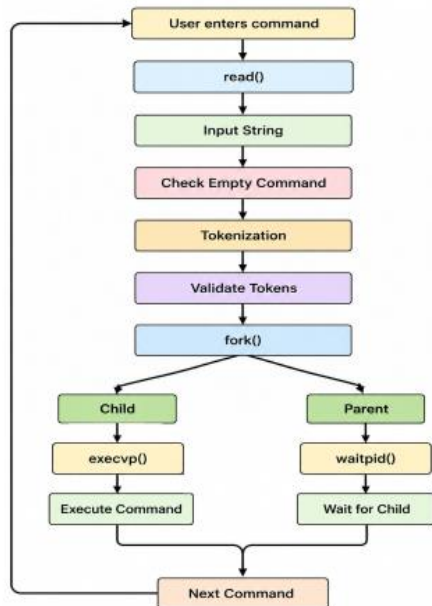


Skill Week 4

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Flow Chart:



```
divya@Divyasree:~$ mkdir skill4
divya@Divyasree:~$ cd skill4
divya@Divyasree:~/skill4$ mkdir task4
divya@Divyasree:~/skill4$ mkdir task1
divya@Divyasree:~/skill4$ cd task1
divya@Divyasree:~/skill4/task1$ pwd
/home/divya/skill4/task1
divya@Divyasree:~/skill4/task1$ nano task1.c
divya@Divyasree:~/skill4/task1$ gcc task1.c -o task1
divya@Divyasree:~/skill4/task1$ ./task1
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

// Split input into tokens
int tokenize(char *input, char *tokens[])
{
    int count = 0;

    char *token = strtok(input, " \\t\\n");

    while (token != NULL && count < MAX_TOKENS - 1)
    {
        tokens[count++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    tokens[count] = NULL;

    return count;
}

// Display tokens for debugging
void displayTokens(char *tokens[], int count)
{
    printf("\n--- Parsed Tokens ---\n");

    for (int i = 0; i < count; i++)
    {
        printf("Token %d: %s\n", i + 1, tokens[i]);
    }
}

// Validate token stream
int validateTokens(char *tokens[], int count)
{
    if (count == 0)
    {
        printf("Empty command.\n");
        return 0;
    }

    // Check invalid command structures
    for (int i = 0; i < count; i++)
    {
        if (strcmp(tokens[i], "|") == 0 ||
            strcmp(tokens[i], ";") == 0)
        {
            if (i == 0 || i == count - 1)
            {
                printf("Syntax Error: Invalid command structure.\n");
                return 0;
            }
        }
    }

    return 1;
}

// Display execution structure
void displayExecutionStructure(char *tokens[], int count)
{
    printf("\n--- Execution Structure ---\n");

    printf("Command: %s\n", tokens[0]);

    for (int i = 1; i < count; i++)
    {
        printf("Argument %d: %s\n", i, tokens[i]);
    }
}

```

```

char *tokens[MAX_TOKENS];

printf("=====\n");
printf("      OS COMMAND PARSER\n");
printf("=====\n");

while (1)
{
    printf("\nmyShell> ");
    fflush(stdout);

    // Read user input
    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        break;
    }

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

    // Handle empty command
    if (strlen(input) == 0)
    {
        printf("Empty command. Please enter a command.\n");
        continue;
    }

    // Tokenize input
    int count = tokenize(input, tokens);

    // Validate tokens
    if (!validateTokens(tokens, count))
    {
        continue;
    }

    // Display parsing result
    displayTokens(tokens, count);

    // Display execution structure
    displayExecutionStructure(tokens, count);

    // Create child process
    pid_t pid = fork();

    // Create child process
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        continue;
    }

    // Child process
    if (pid == 0)
    {
        printf("\nChild Process PID: %d\n", getpid());

        printf("Executing command...\n");

        execvp(tokens[0], tokens);

        // If execvp fails
        perror("Execution Error");
        exit(EXIT_FAILURE);
    }

    // Parent process
    else
    {
        int status;

        printf("\nParent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
    }
}

return 0;
}

```

```
=====
OS COMMAND PARSER
=====
```

```
myShell> ls
```

```
--- Parsed Tokens ---
```

```
Token 1: ls
```

```
--- Execution Structure ---
```

```
Command: ls
```

```
Parent Process PID: 564
```

```
Created Child PID: 565
```

```
Child Process PID: 565
```

```
Executing command...
```

```
task1 task1.c
```

```
Child exited with status: 0
```

```
myShell> echo Hello World
```

```
--- Parsed Tokens ---
```

```
Token 1: echo
```

```
Token 2: Hello
```

```
Token 3: World
```

```
--- Execution Structure ---
```

```
Command: echo
```

```
Argument 1: Hello
```

```
Argument 2: World
```

```
Parent Process PID: 564
```

```
Created Child PID: 565
```

```
Child Process PID: 565
```

```
Executing command...
```

```
task1 task1.c
```

```
Child exited with status: 0
```

```
myShell> echo Hello World
```

```
--- Parsed Tokens ---
```

```
Token 1: echo
```

```
Token 2: Hello
```

```
Token 3: World
```

```
--- Execution Structure ---
```

```
Command: echo
```

```
Argument 1: Hello
```

```
Argument 2: World
```

```
Parent Process PID: 564
```

```
Created Child PID: 568
```

```
Child Process PID: 568
```

```
Executing command...
```

```
Hello World
```

```
Child exited with status: 0
```

```
myShell> echo      Hello      OS      World
```

```
--- Parsed Tokens ---
```

```
Token 1: echo
```

```
Token 2: Hello
```

```
--- Parsed Tokens ---
Token 1: echo
Token 2: Hello
Token 3: OS
Token 4: World

--- Execution Structure ---
Command: echo
Argument 1: Hello
Argument 2: OS
Argument 3: World

Parent Process PID: 564
Created Child PID: 575

Child Process PID: 575
Executing command...
Hello OS World
Child exited with status: 0

myShell>
Empty command. Please enter a command.

myShell> |
Syntax Error: Invalid command structure.

--- Execution Structure ---
Command: abcxyz

Parent Process PID: 564

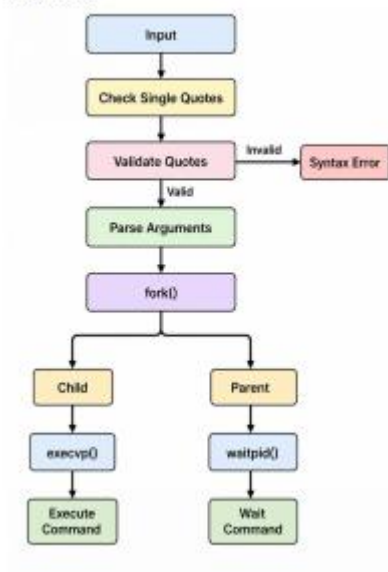
Child Process PID: 584
Created Child PID: 584
Executing command...
Execution Error: No such file or directory
Child exited with status: 1

myShell> ^C
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Flow Chart:



```
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100
#define MAX_TOKEN 1024

// Parse input and handle single quotes
int parseCommand(char *input, char *args[])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        // Skip whitespace outside quotes
        while (input[i] == ' ' || input[i] == '\t' || input[i] == '\n')
        {
            i++;
        }

        if (input[i] == '\0')
            break;

        char *token = malloc(MAX_TOKEN);
        int j = 0;
        int singleQuote = 0;

        // Parse one argument
        while (input[i] != '\0')
        {
            // Start or end single quote
            if (input[i] == '\'' || input[i] == '"')
            {
                singleQuote = !singleQuote;
                i++;
                continue;
            }

            // Whitespace outside quotes ends the argument
            if (!singleQuote && (input[i] == ' ' || input[i] == '\t' || input[i] == '\n'))
            {
                break;
            }

            // Store character literally
            token[j++] = input[i];
        }
    }
}
```

```

        }
        break;
    }

    // Store character literally
    token[j++] = input[i];
    i++;
}

token[j] = '\0';

// Check for unmatched single quote
if (singleQuote)
{
    printf("\nSyntax Error: Unmatched single quote.\n");
    free(token);

    // Free previously allocated arguments
    for (int k = 0; k < argc; k++)
    {
        free(args[k]);
    }

    return -1;
}

args[argc++] = token;

// Skip whitespace before next argument
while (input[i] == ' ' ||
        input[i] == '\t' ||
        input[i] == '\n')
{
    i++;
}

if (argc >= MAX_ARGS - 1)
    break;
}

args[argc] = NULL;
return argc;
}

// Display parsed arguments
void displayArguments(char *args[], int argc)
{
    printf("\n--- Parsing Result ---\n");

    for (int i = 0; i < argc; i++)
    {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }
}

void displayArguments(char *args[], int argc)
{
    printf("\n--- Parsing Result ---\n");

    for (int i = 0; i < argc; i++)
    {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("===== \n");
    printf("SINGLE QUOTE PARSER\n");
    printf("===== \n");

    while (1)
    {
        printf("\nnmyShell> ");
        fflush(stdout);

        // Read input
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Handle empty command
        if (strlen(input) == 0)
        {
            printf("Empty command. Please enter a command.\n");
            continue;
        }

        // Parse command
        int argc = parseCommand(input, args);

        // Syntax error
        if (argc == -1)
        {
            continue;
        }

        // Empty command after parsing
        if (argc == 0)
        {
            printf("Empty command. Please enter a command.\n");
        }
    }
}

```

```

// Empty command after parsing
if (argc == 0)
{
    printf("Empty command. Please enter a command.\n");
    continue;
}

// Display parsing result
displayArguments(args, argc);

// Create child process
pid_t pid = fork();
if (pid < 0)
{
    perror("fork failed");
    for (int i = 0; i < argc; i++)
        free(args[i]);
    continue;
}

// Child process
if (pid == 0)
{
    printf("\nChild PID: %d\n", getpid());
    execvp(args[0], args);
    perror("Execution Error");
    exit(EXIT_FAILURE);
}

// Parent process
else
{
    int status;
    printf("\nParent PID: %d\n", getpid());
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
    {
        printf("\nCommand execution completed.\n");
    }
}

// Free allocated arguments
for (int i = 0; i < argc; i++)
{
    free(args[i]);
}
}

```

divya@Divyasree:~/skill4/task2\$ nano task2.c

divya@Divyasree:~/skill4/task2\$ gcc task2.c -o task2

divya@Divyasree:~/skill4/task2\$./task2

```

=====
SINGLE QUOTE PARSER
=====

```

myShell> echo Hello

--- Parsing Result ---

Argument 1: echo

Argument 2: Hello

Parent PID: 665

Child PID: 666

Hello

Command execution completed.

myShell> echo 'Hello World'

```
=====
SINGLE QUOTE PARSER
=====

myShell> echo Hello

--- Parsing Result ---
Argument 1: echo
Argument 2: Hello

Parent PID: 665

Child PID: 666
Hello

Command execution completed.

myShell> echo 'Hello World'

--- Parsing Result ---
Argument 1: echo
Argument 2: Hello World

Parent PID: 665

Child PID: 1220
Hello World

Command execution completed.

myShell> echo 'Hello $USER'

--- Parsing Result ---

Command execution completed.

myShell> echo 'Hello $USER'

--- Parsing Result ---
Argument 1: echo
Argument 2: Hello $USER

Parent PID: 665

Child PID: 1226
Hello $USER

Command execution completed.

myShell> echo 'Hello World

Syntax Error: Unmatched single quote.

myShell> ^C
divya@Divyasree:~/skill4/task2$ nano task2.c
divya@Divyasree:~/skill4/task2$ nano task2.c
divya@Divyasree:~/skill4/task2$ |
```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
divya@Divyasree:~$ cd skill 4
-bash: cd: too many arguments
divya@Divyasree:~$ cd ~/skill4
divya@Divyasree:~/skill4$ ls
task1 task2 task4
divya@Divyasree:~/skill4$ mkdir task3
divya@Divyasree:~/skill4$ cd task3
divya@Divyasree:~/skill4/task3$ pwd
/home/divya/skill4/task3
divya@Divyasree:~/skill4/task3$ nano task3.c
divya@Divyasree:~/skill4/task3$ gcc task3.c -o task3
divya@Divyasree:~/skill4/task3$ ls
task3 task3.c
divya@Divyasree:~/skill4/task3$ ./task3
```

```
=====
DOUBLE QUOTE PARSER
=====
myShell> echo "Hello World"

--- Parsing Result ---
Argument 1: echo
Argument 2: Hello World
Parent Process PID: 663
Created Child PID: 666

Child Process
PID = 666
Hello World
Child process completed.
myShell> echo "Hello $USER"

--- Parsing Result ---
Argument 1: echo
Argument 2: Hello divya
Parent Process PID: 663
Created Child PID: 667

Child Process
PID = 667
Hello divya
Child process completed.
```

```

myShell> echo "Hello OS World"

--- Parsing Result ---
Argument 1: echo
Argument 2: Hello OS World
Parent Process PID: 663
Created Child PID: 668

Child Process
PID = 668
Hello OS World
Child process completed.
myShell> echo "Hello World
Syntax Error: Unmatched double quote.
myShell> |

```

Source code:

```

GNU nano 7.2 /usr/src/.../task3.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int parseDoubleQuotes(char *input, char *args[])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        if (input[i] == '\0')
            break;

        if (input[i] == '"')
        {
            i++;

            char buffer[MAX_INPUT];
            int j = 0;
            int closed = 0;

            while (input[i] != '\0')
            {
                if (input[i] == '"')
                {
                    closed = 1;
                    i++;
                    break;
                }
            }
        }
    }
}

```

```

        buffer[j++] = input[i++];
    }

    if (!closed)
    {
        printf("Syntax Error: Unmatched double quote.\n");
        return -1;
    }

    buffer[j] = '\0';

    args[count] = malloc(strlen(buffer) + 1);
    strcpy(args[count], buffer);
    count++;
}
else
{
    char buffer[MAX_INPUT];
    int j = 0;

    while (input[i] != '\0' &&
           input[i] != ' ' &&
           input[i] != '\t')
    {
        buffer[j++] = input[i++];
    }

    buffer[j] = '\0';

    args[count] = malloc(strlen(buffer) + 1);
    strcpy(args[count], buffer);
    count++;
}

if (count >= MAX_ARGS - 1)
    break;
}

args[count] = NULL;
return count;
}

```

```

void expandVariables(char *args[], int count)
{
    for (int i = 0; i < count; i++)
    {
        char result[MAX_INPUT];
        int j = 0;
        int k = 0;

        while (args[i][j] != '\0')
        {
            if (args[i][j] == '$' &&
                args[i][j + 1] != '\0')
            {
                char variable[100];
                int v = 0;

                j++;

                while (args[i][j] != '\0' &&
                       ((args[i][j] >= 'A' && args[i][j] <= 'Z') ||
                        (args[i][j] >= 'a' && args[i][j] <= 'z') ||
                        (args[i][j] >= '0' && args[i][j] <= '9') ||
                        args[i][j] == '_'))
                {
                    variable[v++] = args[i][j++];
                }

                variable[v] = '\0';

                char *value = getenv(variable);

                if (value != NULL)
                {
                    for (int x = 0; value[x] != '\0'; x++)
                        result[k++] = value[x];
                }
            }
        }
    }
}

```

```

        else
        {
            result[k++] = args[i][j++];
        }
    }

    result[k] = '\0';

    free(args[i]);

    args[i] = malloc(strlen(result) + 1);
    strcpy(args[i], result);
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("      DOUBLE QUOTE PARSER\n");
    printf("=====\n");

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            printf("Empty command. Please enter a command.\n");
            continue;
        }

        int count = parseDoubleQuotes(input, args);

        if (count == -1)
            continue;

        if (count == 0)
            continue;

        expandVariables(args, count);

        printf("\n--- Parsing Result ---\n");

        for (int i = 0; i < count; i++)
        {
            printf("Argument %d: %s\n", i + 1, args[i]);
        }

        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            continue;
        }

        if (pid == 0)
        {
            printf("\nChild Process\n");
            printf("PID = %d\n", getpid());

            execvp(args[0], args);

            perror("Execution Error");
            exit(EXIT_FAILURE);
        }
        else
        {
            int status;

            int status;

            printf("Parent Process PID: %d\n", getpid());
            printf("Created Child PID: %d\n", pid);

            waitpid(pid, &status, 0);

            printf("Child process completed.\n");
        }

        for (int i = 0; i < count; i++)
            free(args[i]);
    }

    return 0;
}

```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
divya@Divyasree:~/skill4/task3$ cd ..
divya@Divyasree:~/skill4$ cd task4
divya@Divyasree:~/skill4/task4$ pwd
/home/divya/skill4/task4
divya@Divyasree:~/skill4/task4$ nano task4.c
divya@Divyasree:~/skill4/task4$ gcc task4.c -o task4
divya@Divyasree:~/skill4/task4$ ./task4
```

```
=====
      ESCAPE SEQUENCE PARSER
=====
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Parent Process PID: 807
Created Child PID: 809

Child Process PID: 809
Hello
Child exited with status: 0
myShell> echo Hello\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Parent Process PID: 807
Created Child PID: 810

Child Process PID: 810
Hello World
Child exited with status: 0
```

```
myShell> echo Hello\ OS\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello OS World
Parent Process PID: 807
Created Child PID: 811

Child Process PID: 811
Hello OS World
Child exited with status: 0
myShell> echo Hello\!

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello!
Parent Process PID: 807
Created Child PID: 812

Child Process PID: 812
Hello!
Child exited with status: 0
myShell> |
```

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int parseCommand(char *input, char *args[])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        if (input[i] == '\0')
            break;

        char buffer[MAX_INPUT];
        int j = 0;

        while (input[i] != '\0' &&
               input[i] != ' ' &&
               input[i] != '\t')
        {
            if (input[i] == '\\')
            {
                i++;

                if (input[i] != '\0')
                {
                    buffer[j++] = input[i++];
                }
            }
            else
            {
                buffer[j++] = input[i++];
            }
        }

        buffer[j] = '\0';

        args[count] = malloc(strlen(buffer) + 1);
        strcpy(args[count], buffer);
        count++;

        if (count >= MAX_ARGS - 1)
            break;
    }

    args[count] = NULL;

    return count;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("      ESCAPE SEQUENCE PARSER\n");
    printf("=====\n");

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            printf("Empty command. Please enter a command.\n");
            continue;
        }

        int count = parseCommand(input, args);

        if (count == 0)
            continue;
    }
}
```

```

printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}

pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
    continue;
}

if (pid == 0)
{
    printf("\nChild Process PID: %d\n", getpid());

    execvp(args[0], args);

    perror("Execution Error");
    exit(EXIT_FAILURE);
}
else
{
    int status;

    printf("Parent Process PID: %d\n", getpid());
    printf("Created Child PID: %d\n", pid);

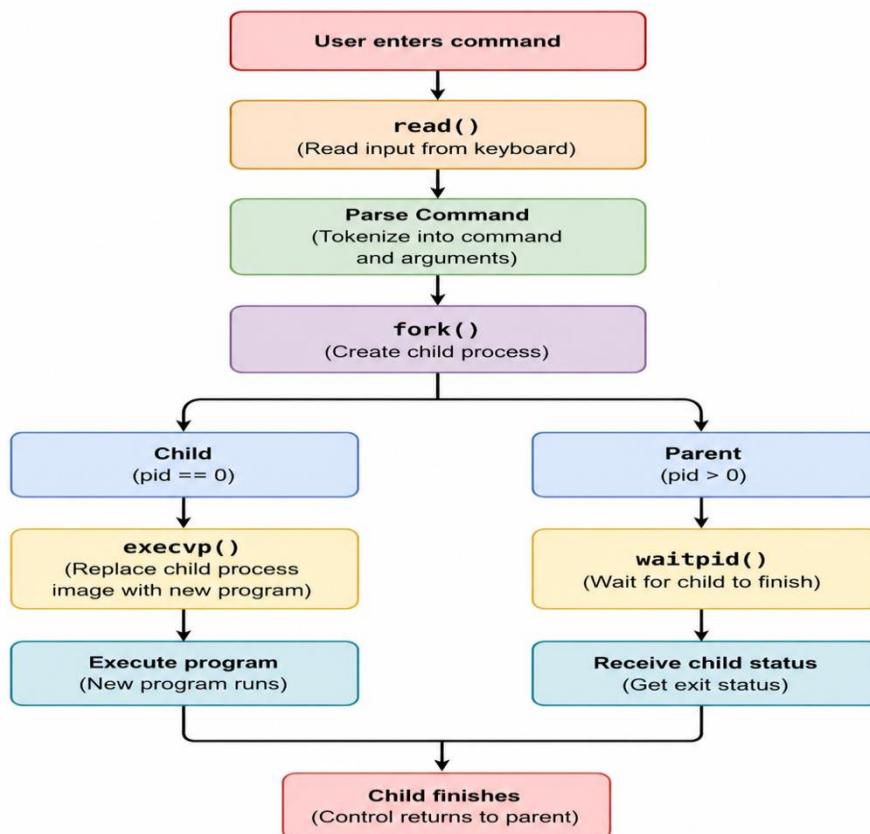
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child exited with status: %d\n",
            WEXITSTATUS(status));
    }
}

for (int i = 0; i < count; i++)
{
    free(args[i]);
}

```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.
Flow Chart:



```
divya@Divyasree:~/skill4/task4$ cd ..
divya@Divyasree:~/skill4$ mkdir task5
divya@Divyasree:~/skill4$ cd task5
divya@Divyasree:~/skill4/task5$ pwd
/home/divya/skill4/task5
divya@Divyasree:~/skill4/task5$ nano task5.c
divya@Divyasree:~/skill4/task5$ gcc task5.c -o task5
divya@Divyasree:~/skill4/task5$ ls
task5  task5.c
divya@Divyasree:~/skill4/task5$ ./task5
```

```
=====
COMMAND EXECUTION
=====
myshell$ ls
Parent Process PID: 857
Created Child PID: 858
Child Process PID: 858
task5  task5.c
Child exited with status: 0
myshell$ ls -l
Parent Process PID: 857
Created Child PID: 861
Child Process PID: 861
total 24
-rwxr-xr-x 1 divya divya 16568 Aug 24 05:38 task5
-rw-r--r-- 1 divya divya 1651 Aug 24 05:38 task5.c
Child exited with status: 0
myshell$ pwd
Parent Process PID: 857
Created Child PID: 862
Child Process PID: 862
/home/divya/skill4/task5
Child exited with status: 0
myshell$ abcdxyz
Parent Process PID: 857
Created Child PID: 863
Child Process PID: 863
Execution Error: No such file or directory
Child exited with status: 1
```

```
myshell$ ls @l
Parent Process PID: 857
Created Child PID: 865
Child Process PID: 865
ls: cannot access '@l': No such file or directory
Child exited with status: 2
myshell$ |
```

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("      COMMAND EXECUTION\n");
    printf("=====\n");

    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        int count = 0;
        char *token = strtok(input, " \t");

        while (token != NULL && count < MAX_ARGS - 1)
        {
            args[count++] = token;
            token = strtok(NULL, " \t");
        }

        args[count] = NULL;

        if (count == 0)
            continue;

        printf("Parent Process PID: %d\n", getpid());

        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            continue;
        }

        if (pid == 0)
        {
            printf("Child Process PID: %d\n", getpid());

            execvp(args[0], args);

            perror("Execution Error");
            exit(1);
        }
        else
        {
            printf("Created Child PID: %d\n", pid);

            int status;

            waitpid(pid, &status, 0);

            if (WIFEXITED(status))
            {
                printf("Child exited with status: %d\n",
                    WEXITSTATUS(status));
            }
        }
    }

    return 0;
}
```